Real-Time Virtual Mentor for Introductory Programming Feedback

Alimzhan Koshikov
Master Degree Student, School of Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
225148@astanait.edu.kz
0009-0003-3234-6487

Tutkyshbayeva Shyryn
PhD, School of Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
sh.tutkyshbayeva@astanait.edu.kz
0000-0002-1419-2571

***Abstract*—Introductory programming courses often provide feedback that is either delayed, overly generic, or limited to correctness checks. As a result, first-year students frequently repeat the same syntax and logic mistakes without receiving guidance that is timely or pedagogically appropriate. This paper presents a virtual mentor for CS1 that delivers real-time, personalized feedback during active problem solving. The proposed system combines continuous code analysis, a lightweight student model, and a pedagogical policy layer that determines not only *what* feedback to provide, but also *when* and *how much* help should be given. Unlike unconstrained conversational assistants, the mentor regulates feedback disclosure and selects among guiding questions, conceptual hints, code highlighting, or no feedback, depending on the current error state and its recurrence. A pilot log-based evaluation of the prototype showed that the system maintained interactive responsiveness, with a mean end-to-end latency of 741 ms and 809 ms at P95. Across logged events, the prototype achieved an event-level success rate of 66.7% and a median time-to-fix of 2.45 s. These results indicate that a bounded-latency, pedagogically constrained feedback pipeline is feasible as a support tool for CS1 instruction.**

***Index Terms*—Virtual mentor, CS1, introductory programming, programming education, real-time feedback, student modeling, pedagogical constraints**

I. Introduction

Introductory programming (CS1) remains difficult for many first-year students because they must learn syntax, program logic, and problem decomposition at the same time. In large classes, beginners often move through cycles of compilation failures and repeated misconceptions with limited access to individualized help. For this reason, timely formative feedback is important, yet providing it consistently at scale remains difficult for instructors and teaching assistants [1], [2].

Automated assessment improves scalability through unit tests, static analysis, and compiler diagnostics, but such feedback is often binary and does not adequately address why an error occurred or what kind of support a learner needs next. Prior work in computing education shows that feedback quality depends not only on correctness, but also on timing, pedagogical framing, and whether the support preserves student autonomy instead of simply accelerating task completion [1], [3]. Recent large-scale AI tutoring deployments demonstrate that near real-time support is technically feasible. For example, CS50 reported substantial use of its AI chatbot in 2024, reaching about 211,000 learners and roughly 10 million queries by mid-November, with an estimated cost of about $1.50 per student per year [4]. At the same time, maintaining pedagogical alignment and reliability requires continuous monitoring and evaluation [4]. Randomized and classroom studies in CS1 also suggest that more interactive feedback does not automatically lead to better learning outcomes, especially when students become over-reliant on the tool or receive inaccurate explanations [2], [3], [5].

These observations point to a practical gap. CS1 learners need support that is available during coding, adapts to recurring error patterns, and remains pedagogically constrained. Although prior systems such as CodeAid and Iris introduce guardrails [6], [7], many LLM-based assistants still operate as general-purpose helpers without explicit learner modeling, while intelligent tutoring systems often require heavier domain engineering or do not focus on low-latency intervention during code construction [8]. In response, this paper presents a *virtual mentor* for CS1 that combines continuous code analysis, a lightweight student model, and a pedagogical policy layer for bounded-latency feedback selection. The goal is not to replace instructors or generate unrestricted answers, but to deliver the right level of guidance at the right moment while avoiding direct solution disclosure.

The main contributions of this work are as follows:

- a modular virtual mentor architecture that combines real-time code analysis, lightweight student modeling, and pedagogical control for CS1 support;
- a feedback mechanism that adapts guidance style and granularity to recurring error patterns while avoiding direct solution disclosure;
- a pilot prototype evaluation based on event-level interaction logs, reporting latency characteristics and feedback effectiveness trends under real-time use.

II. Related Work

Research on AI support for programming education commonly falls into three broad directions: intelligent tutoring systems, automated feedback generation, and LLM-based assistants. The proposed virtual mentor draws from all three, but

combines them with a different emphasis on bounded-latency interaction and policy-level control.

A. Intelligent Tutoring Systems for Programming

Intelligent tutoring systems (ITS) personalize support by modeling learner state and adapting feedback or task sequencing. Recent systems increasingly aim to augment instructors rather than fully automate instruction. ProgTutor, for example, combines adaptive sequencing with automated evaluation while keeping instructors in the loop [8]. A common limitation, however, is that many ITS intervene at discrete checkpoints or after submission, which reduces their usefulness during the moment-to-moment process of code construction.

The proposed virtual mentor targets this in-process setting by monitoring code continuously and selecting interventions during active problem solving rather than only after a completed attempt.

B. Automated Feedback Generation Systems

Automated feedback is widely used in CS1 because it scales well and is easy to integrate into programming courses. Typical implementations rely on unit tests, static analysis, or compiler messages. These methods are useful for fast validation, but they often remain shallow: they say whether something is wrong without adapting the explanation to the learner's misunderstanding. Kumar et al. note that many automated feedback systems emphasize syntax correction or output correctness rather than concept-level guidance, while reproducibility and personalization remain open challenges [9].

This limitation motivates the combination used in our work: code analysis alone is not sufficient, so it is coupled with lightweight student modeling and a pedagogical decision layer.

C. LLM-Based Programming Assistants

Compared with rule-based systems, LLM-based assistants are more flexible in generating explanations and hints in natural language [3]. Systems such as CodeAid and Iris show that LLM-supported assistance can be valuable when pedagogical constraints are built into the interaction design [6], [7]. At the same time, classroom evidence highlights important risks. Students may seek immediate answers instead of engaging with the underlying concept, and the assistant may still produce explanations that are inaccurate, overly strong, or poorly aligned with course objectives [2], [5]. At larger scale, CS50 also emphasizes the need for continuous human feedback and systematic evaluation to keep AI support pedagogically aligned [4].

D. Research Gap and Positioning

Existing approaches rarely combine three properties in one system: real-time support during code writing, an explicit lightweight student model that tracks recurring mistakes, and a pedagogical policy that regulates feedback timing and granularity. Many LLM assistants function as external help tools without explicit learner state, while ITS platforms provide richer personalization but may not be optimized for low-latency intervention during live coding. In addition, recent surveys point to limited reproducibility, private datasets, and the continued risk of hallucinated educational feedback in LLM-based systems [9].

The proposed virtual mentor addresses this gap by combining continuous code analysis, lightweight student modeling, and a policy layer that controls disclosure and timing. The system is positioned not as a general conversational tutor, but as a bounded real-time support mechanism for CS1 that aims to reduce repeated errors while preserving learner autonomy.

Table I summarizes representative systems and positions the proposed mentor relative to prior work.

III. Virtual Mentor System Architecture

The virtual mentor is designed as a bounded real-time decision pipeline that maps a stream of student code states to pedagogically constrained feedback actions. Its architecture separates code analysis, student modeling, policy decision, and feedback realization in order to preserve modularity, interpretability, and latency control.

Unlike standalone LLM assistants used as external help tools, the mentor is embedded directly into the programming workflow and incrementally observes code edits. If LLM-based text generation is used, it serves only as a realization component; the decision about whether feedback should be given, and in what form, is made by the pedagogical policy. This design follows prior work arguing that effective AI teaching assistants must balance immediacy, personalization, and learner autonomy while avoiding direct solution disclosure [1], [6].

A. Data Flow

As shown in Fig. 1, code edits are captured in real time by an analyzer that performs syntactic checks, lightweight structural analysis, and detection of common novice error patterns. The extracted error type and code-level features are then passed to a profile manager that maintains a lightweight learner state, including recurring mistakes, prior feedback exposure, and approximate progression. Based on this state, the policy layer selects an action, and the feedback engine realizes the final response. To preserve interactivity, the full analysis–decision–feedback cycle is executed under bounded response time.

B. Formal System Model

Let c_t be the student code state after a modification at time t. The analyzer

$$A : c_t \to e_t$$

maps c_t to an error type $e_t \in \mathcal{E}$, where $\mathcal{E}$ is a finite set of novice error categories.

The student profile is represented as a lightweight state vector

$$s_t = (n_{e_1}, n_{e_2}, \ldots, n_{e_k}),$$

where n_{e_i} is the cumulative count of error type e_i observed up to time t.

Given (e_t, s_t), the pedagogical policy selects an action

$$\pi : (e_t, s_t) \to a_t,$$

TABLE I
COMPARATIVE ANALYSIS OF AUTOMATED PROGRAMMING FEEDBACK SYSTEMS

System	Technology	Feedback Type	Effectiveness Metric
AIF 3.0 [10]	Hybrid (data-driven + expert-based)	Progress indicators (0–100%)	Subgoal detection accuracy: 88%
CodeAid [6]	GPT-3.5-Turbo	Pseudocode generation and explanations	Technical correctness: 79%
Verifix [9]	Program analysis (CFA)	Verified repair suggestions	Feedback quality score: 3.8/5.0
DeepFix [9]	Machine learning (neural networks)	Syntax error correction	Full error correction rate: 27%
Proposed model	Rule-based baseline + optional learned policy + student modeling	Real-time personalized feedback with pedagogical constraints	Pilot results reported in Sec. V (Table II)

where $a_t \in \mathcal{A}$ and $\mathcal{A} = \{\text{GQ}, \text{CH}, \text{HL}, \text{NF}\}$ denote *Guiding Question*, *Conceptual Hint*, *Code Highlight*, and *No Feedback*.

In the baseline configuration, π is rule-based. In the adaptive configuration, π is learned from interaction logs through supervised multi-class classification:

$$\hat{a} = \arg\max_{a \in \mathcal{A}} P(a \mid x),$$

where x includes the current error type, recurrence counts, and timing-related interaction features.

The system does not train or fine-tune LLMs. Machine learning is applied only to policy learning, while language generation, if used, remains a constrained realization step.

C. Core Components

Analyzer: Continuously evaluates code using syntactic validation, structural checks, and novice error detection.

Profile Manager: Maintains an interpretable learner state with low computational overhead, prioritizing scalability over heavy knowledge tracing [8].

Policy Layer: Selects feedback actions using explicit pedagogical rules or a learned policy trained on interaction logs.

Feedback Engine: Realizes the selected action as a guiding question, conceptual hint, or code highlight while avoiding direct solution disclosure [1], [6].

IV. REAL-TIME FEEDBACK MECHANISM

The feedback mechanism is intended to support beginners during active problem solving rather than only after task completion. Prior studies on LLM-based programming assistants show that feedback which is delayed, overly explicit, or poorly timed can encourage trial-and-error behavior and reduce conceptual engagement [5]. For this reason, the proposed system generates feedback incrementally as students modify their code.

A. Feedback Generation

When a student writes or edits code, the analyzer first detects syntax and logic-related patterns associated with novice mistakes. The resulting signals are then stored in the student profile, allowing the system to distinguish between a new issue and a repeated one. Based on this context, the policy layer chooses an appropriate feedback strategy, such as a conceptual hint, a guiding question, or a code highlight.

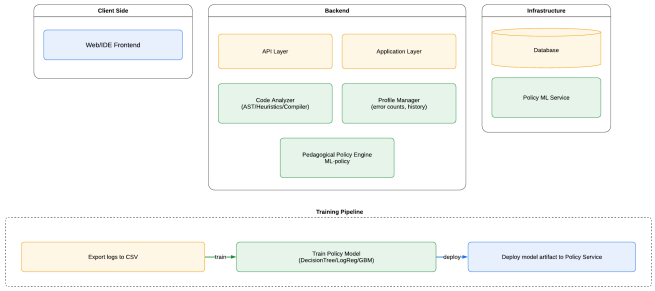

Fig. 1. High-level architecture of the virtual mentor system.

This design follows prior findings that hints are often more educationally useful than direct solutions because they help students remain engaged with the problem while still receiving support [1], [7]. Accordingly, the system intentionally avoids revealing complete solutions.

B. Timing Constraints

To support real-time interaction, the full feedback cycle relies on lightweight analysis and incremental updates of the student model. This makes it possible to respond immediately after a code modification without introducing heavy inference delays.

The total response time is defined as

$$T_{total} = T_{analysis} + T_{policy} + T_{feedback},$$

where $T_{analysis}$ denotes code analysis time, T_{policy} is pedagogical decision time, and $T_{feedback}$ represents feedback realization time.

In the prototype, the interaction logs indicate that the pipeline remained within interactive time bounds, with mean $T_{total} =$ **741** ms and P95 **809** ms. Average component times were **618** ms for analysis, **26** ms for policy selection, and **12** ms for feedback realization.

V. EXPERIMENTAL EVALUATION

A. Evaluation Setup

The current evaluation is a pilot prototype study based on event-level interaction logs collected during system use on

TABLE II
PILOT RESULTS FROM INTERACTION LOGS (EVENT-LEVEL)

Metric	Value
Events (N)	60
Mean latency $\bar{T}$ (ms)	741
P95 latency (ms)	809
Mean analysis/policy/feedback (ms)	618 / 26 / 12
Platform overhead (ms)	85
Event success rate (%)	66.7
Median time-to-fix (ms)	2450
P95 time-to-fix (ms)	3595

TABLE III
SUCCESS RATE BY FEEDBACK ACTION (PILOT)

Action	N	Success rate
CODE_HIGHLIGHT	23	1.00
CONCEPTUAL_HINT	15	1.00
GUIDING_QUESTION	19	0.25
NO_FEEDBACK	3	0.00

introductory programming tasks. The purpose of this evaluation is not to claim full classroom effectiveness, but to assess whether the proposed pipeline can operate within real-time constraints and whether different feedback actions show useful resolution trends at the event level.

Each interaction record is represented as

$$x_i = (e_i, c_i, t_i^{analysis}, t_i^{feedback}, s_i),$$

where e_i denotes the detected error type, c_i the recurrence count, $t_i^{analysis}$ and $t_i^{feedback}$ the measured latency components, and s_i the outcome indicator showing whether the issue was resolved after the feedback event.

These logs form a structured pilot dataset for analyzing responsiveness and action-level effectiveness. They also provide the basis for future policy learning.

B. Metrics

The prototype was evaluated using operational and event-level effectiveness metrics. Average system response latency was measured as

$$\bar{T} = \frac{1}{N} \sum_{i=1}^{N} T_{total}^{(i)},$$

which captures end-to-end responsiveness during interaction.

In addition, event success rate and time-to-fix were used as feedback effectiveness proxies. In this setting, an event is considered successful when the issue identified for that interaction is resolved after the issued feedback.

C. Results

Table II summarizes pilot outcomes computed from event-level interaction logs. We report (i) responsiveness of the real-time pipeline via latency statistics and (ii) event-level effectiveness proxies through success and time-to-fix.

VI. DISCUSSION

A. Interpretation of Results

The pilot results suggest that the proposed virtual mentor can operate within interactive time constraints while providing useful event-level support. The end-to-end latency remained below one second in both mean and P95 measurements, which indicates that the analysis–policy–feedback pipeline is suitable for real-time use during code construction.

The event-level success rate of **66.7%** and median time-to-fix of **2.45** s suggest that many issues were resolved shortly after feedback was delivered. At the same time, these values should be interpreted as pilot indicators rather than as final evidence of learning impact.

The action-level breakdown in Table III shows that conceptual hints and code highlighting were associated with stronger immediate resolution rates than guiding questions in the current trace. This may indicate that more explicit but still constrained interventions are particularly useful for certain novice errors, while guiding questions may require better policy tuning or richer student state.

More broadly, the results support the central design assumption of the paper: feedback quality in CS1 depends not only on correctness, but also on timing, granularity, and pedagogical framing [1], [3]. A policy-controlled mentor can therefore offer a middle ground between generic automated checks and unrestricted conversational assistance.

B. Limitations

This study has several limitations. First, the current evaluation is pilot-scale and covers a limited number of introductory programming interactions. Second, the results are event-level rather than longitudinal; they do not yet measure retention, transfer, or broader task-level learning outcomes. Third, the present study does not include a full benchmark comparison against external tutoring systems or a classroom-scale controlled experiment. Finally, the student model is intentionally lightweight. Although this improves interpretability and runtime efficiency, it does not capture richer learning dynamics that may become important in longer-term deployments [8], [11].

C. Practical Implications

Despite these limitations, the findings suggest that real-time personalized feedback can be delivered without replacing instructors or teaching assistants. With explicit pedagogical control and lightweight student modeling, the mentor can complement existing automated assessment tools and reduce repetitive support load while preserving instructional intent. In practice, such a system is best viewed as an intermediate layer between static automated checks and human tutoring, rather than as an autonomous substitute for either [4], [6].

D. Ethical Considerations

The system is designed to preserve student autonomy by avoiding direct solution disclosure and prioritizing formative guidance [1]. Interaction logs are anonymized before analysis.

In addition, pedagogical controls are used to reduce over-reliance, which remains a recurring concern in studies of LLM-supported educational tools [5], [6].

VII. Future Work

Future work will proceed in four directions. First, larger-scale evaluations are needed to test whether the observed event-level gains translate into meaningful task-level and longitudinal learning outcomes. Second, the student model can be extended beyond simple recurrence counts to capture temporal learning dynamics while remaining interpretable and computationally lightweight. Third, instructor-facing controls should be added so that feedback policies can be inspected and calibrated for course-specific pedagogical goals. Finally, the approach should be tested across additional languages and programming paradigms to evaluate its generalizability beyond introductory procedural tasks.

VIII. Conclusion

This paper presented a virtual mentor for introductory programming that delivers real-time feedback through continuous code analysis, lightweight student modeling, and pedagogical policy control. Rather than acting as an unrestricted conversational assistant, the system selects bounded and context-aware interventions during active problem solving.

A pilot log-based evaluation showed that the prototype maintained interactive responsiveness, with mean end-to-end latency of **741** ms and P95 latency of **809** ms, while achieving an event-level success rate of **66.7%** and a median time-to-fix of **2.45** s. These findings support the feasibility of bounded-latency, pedagogically constrained feedback pipelines as scalable support tools for CS1. Further work is needed to evaluate the approach on larger cohorts, under comparative baselines, and with longer-term learning measures.

Acknowledgment

This work was carried out as part of the master's degree program at Astana IT University. The authors thank Astana IT University for providing the academic environment and computing infrastructure that supported the development of the prototype.

References

[1] P. Denny, S. MacNeil, J. Savelka, L. Porter, and A. Luxton-Reilly, "Desirable characteristics for ai teaching assistants in programming education," p. 408–414, 2024. [Online]. Available: https://doi.org/10.1145/3649217.3653574

[2] U. Z. Ahmed, S. Sahai, B. Leong, and A. Karkare, "Feasibility study of augmenting teaching assistants with ai for cs1 programming feedback," pp. 11–17, 2025. [Online]. Available: https://doi.org/10.1145/3641554.3701972

[3] A. F. Pereira and R. Ferreira Mello, "A systematic literature review on large language models applications in computer programming teaching evaluation process," *IEEE Access*, vol. 13, pp. 113 449–113 460, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3584060

[4] R. Liu, J. Zhao, B. Xu, C. Perez, Y. Zhukovets, and D. J. Malan, "Improving ai in cs50: Leveraging human feedback for better learning," p. 715–721, 2025. [Online]. Available: https://doi.org/10.1145/3641554.3701945

[5] B. Sheese, M. Liffiton, J. Savelka, and P. Denny, "Patterns of student help-seeking when using a large language model-powered programming assistant," p. 49–57, 2024. [Online]. Available: https://doi.org/10.1145/3636243.3636249

[6] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, "Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs," 2024. [Online]. Available: https://doi.org/10.1145/3613904.3642773

[7] P. Bassner, E. Frankford, and S. Krusche, "Iris: An ai-driven virtual tutor for computer science education," p. 394–400, 2024. [Online]. Available: https://doi.org/10.1145/3649217.3653543

[8] J. Ortega-Morla, A. Leis, A. Mallo, L. Morán-Fernández, S. Guerreiro, A. Paz-López, B. Pérez-Sánchez, N. Sánchez-Maroño, A. Rodríguez-Arias, O. Fontenla-Romero, and F. Bellas, "Progtutor: A robotic-based framework to support teaching and learning of programming fundamentals," *IEEE Transactions on Learning Technologies*, vol. 18, pp. 783–797, 2025. [Online]. Available: https://doi.org/10.1109/TLT.2025.3598041

[9] S. S. Kumar, M. A. Lones, M. Maarek, and H. Zantout, "Navigating the landscape of automated feedback generation techniques for programming exercises," *ACM Trans. Comput. Educ.*, vol. 25, no. 4, Oct. 2025. [Online]. Available: https://doi.org/10.1145/3764593

[10] S. Marwan, B. Akram, T. Barnes, and T. W. Price, "Adaptive immediate feedback for block-based programming: Design and evaluation," *IEEE Transactions on Learning Technologies*, vol. 15, no. 3, pp. 406–420, 2022. [Online]. Available: https://doi.org/10.1109/TLT.2022.3180984

[11] R. Sajja, Y. Sermet, M. Cikmaz, D. Cwiertny, and I. Demir, "Artificial intelligence-enabled intelligent assistant for personalized and adaptive learning in higher education," *Information*, vol. 15, no. 10, 2024. [Online]. Available: https://www.mdpi.com/2078-2489/15/10/596

[12] R. Francisco and F. Silva, "Intelligent tutoring system for computer science education and the use of artificial intelligence: A literature review," p. 338–345, 2022. [Online]. Available: http://dx.doi.org/10.5220/0011084400003182

[13] K. Chrysafiadi, M. Virvou, G. A. Tsihrintzis, and I. Hatzilygeroudis, "Evaluating the user's experience, adaptivity and learning outcomes of a fuzzy-based intelligent tutoring system for computer programming for academic students in greece," *Education and Information Technologies*, vol. 28, no. 6, p. 6453–6483, Nov. 2022. [Online]. Available: http://dx.doi.org/10.1007/s10639-022-11444-3

[14] M. Pankiewicz and R. S. Baker, "Large language models (gpt) for automating feedback on programming assignments," 2023. [Online]. Available: https://arxiv.org/abs/2307.00150

[15] S. Nayak, R. Agarwal, and S. K. Khatri, "Automated assessment tools for grading of programming assignments: A review," pp. 1–4, 2022. [Online]. Available: https://doi.org/10.1109/ICCCI54379.2022.9740769

[16] J.-Y. Kuo, H.-C. Lin, P.-F. Wang, and Z.-G. Nie, "A feedback system supporting students approaching a high-level programming course," *Applied Sciences*, vol. 12, no. 14, 2022. [Online]. Available: https://www.mdpi.com/2076-3417/12/14/7064

[17] I. Estévez-Ayres, P. Callejo, M. Hombrados-Herrera, C. Alario-Hoyos, and C. Delgado Kloos, "Evaluation of llm tools for feedback generation in a course on concurrent programming," p. 774–790, 2024. [Online]. Available: https://doi.org/10.1007/s40593-024-00406-0

[18] T. Phung, V.-A. Pădurean, A. Singh, C. Brooks, J. Cambronero, S. Gulwani, A. Singla, and G. Soares, "Automating human tutor-style programming feedback: Leveraging gpt-4 tutor model for hint generation and gpt-3.5 student model for hint validation," p. 12–23, 2024. [Online]. Available: https://doi.org/10.1145/3636555.3636846

[19] E. Frankford, C. Sauerwein, P. Bassner, S. Krusche, and R. Breu, "Ai-tutoring in software engineering education," p. 309–319, 2024. [Online]. Available: https://doi.org/10.1145/3639474.3640061

[20] M. Messer, N. C. C. Brown, M. Kölling, and M. Shi, "Automated grading and feedback tools for programming education: A systematic review," *ACM Trans. Comput. Educ.*, vol. 24, no. 1, Feb. 2024. [Online]. Available: https://doi.org/10.1145/3636515